

Index

1. Introduction	(1)
➤ 1.1 Project Title	(1)
➤ 1.2 Project Overview	(1)
➤ 1.3 Problem Statement	(1)
➤ 1.4 Objectives	(2)
➤ 1.5 Scope of the Project	(2)
2. Functional Requirements (FR)	(3)
➤ 2.1 Core System Features (FR-01 to FR-10)	(3-4)
3. Non-Functional Requirements (NFR)	(5)
➤ 3.1 Quality & Performance Metrics (NFR-01 to NFR-06)	(5)
4. User Roles & Responsibilities	(6)
➤ 4.1 End-User / Client	(6)
5. Use Cases	(6)
➤ 5.1 Use Case Descriptions	(6-7)

1. Introduction

1.1 Project Title: AI Expense Analyzer

1.2 Project Overview

The AI Expense Analyzer is an advanced personal finance management application developed using the Flutter framework for the frontend, Node.js and Express.js for the backend, and MongoDB for data persistence. Unlike traditional, passive ledgers, this system integrates state-of-the-art machine learning capabilities via the Gemini API to bridge the gap between simple financial tracking and contextual financial management.

The system transforms raw transaction entry into actionable financial intelligence by providing real-time tailored advice, automated transactional categorization, predictive budgeting patterns, and dynamic pattern analysis to help users attain long-term fiscal discipline.

1.3 Problem Statement

Individuals today face significant challenges in maintaining financial discipline due to the complexity of daily transaction tracking and the lack of organized financial data. Most users rely on manual record-keeping or simple, static ledger apps that require significant time and effort, leading to inconsistent data entry and incomplete financial visibility.

Financial stability is fundamental to personal well-being; without a clear understanding of cash flow, users often face unmanaged debt, impulsive overspending, and an inability to save for future goals. Poor financial planning in the present leads to long-term economic instability, making it critical to have a tool that transforms passive data into actionable financial intelligence.

While conventional expense trackers or traditional manual ledgers exist, they primarily serve as digital accounting books. They fail to provide deep contextual insights; they do not automatically flag patterns of waste, nor do they offer proactive, personalized budget recommendations based on the user's unique lifestyle or shifting economic conditions.

Most current solutions require users to manually interpret their own data, which severely limits the effectiveness of the tracking process.

1.4 Objectives

The primary goals of the AI Expense Analyzer system are to transition from basic financial tracking to intelligent financial management through the following objectives:

Develop an Intelligent Financial Tracking Platform: To build a robust and user-friendly system using Flutter that enables seamless recording, categorizing, and visualizing of daily income and expenses.

Implement AI-Driven Financial Analysis: To integrate advanced AI models (via Gemini API) that perform deep-level analysis on raw transaction data to identify spending trends, identify habitual patterns of overspending, and provide actionable financial insights.

Enable Predictive Budgeting and Planning: To leverage AI capabilities to predict future financial outflows based on historical data, allowing users to plan their budgets proactively rather than reacting to expenses after they occur.

Provide Personalized Financial Recommendations: To offer customized, real-time advice tailored to each user's specific financial behavior, helping them optimize savings, reduce non-essential costs, and achieve long-term fiscal discipline.

Enhance Financial Decision-Making: To empower users with comprehensive dashboards and data-driven reports that simplify complex financial data, ultimately fostering improved financial literacy and healthier spending habits.

1.5 Scope of the Project

The scope of the AI Expense Analyzer encapsulates a secure, cross-platform mobile ecosystem tailored for individual consumers. It encompasses backend frameworks for robust data handling and transaction parsing, realtime reporting via clean visualizations, and strict database architectures mapping out personal logs. Crucially, the operational boundaries include automated classification algorithms and standard prompt structures hitting the Gemini cloud interface. The application will explicitly not handle direct automated banking sync/API operations or investment micro-trading features within this release cycle, keeping focus entirely on user-driven logging and automated contextual forecasting.

2. Functional Requirements

Functional requirements specify the core functionalities, procedures, and computations that the system must execute. Each functional requirement is uniquely classified below:

ID	Requirement Name	Description
FR-01	User Registration	The system must allow new users to register a unique account using a valid email address and secure password combination.
FR-02	Secure User Login	The system must authenticate registered users securely using token-based (JWT) mechanisms, supporting secure login sessions.
FR-03	Intelligent Transaction Logger	Users must be able to log manual inputs of income and expenses, containing amount, date, description, and subcategories.
FR-04	Automated Classification	The system must utilize localized processing or automated structures to pre-classify a transaction category based on textual descriptions entered by the user.
FR-05	AI Spending Trend Insight	The system must package historical transaction tables and securely transmit them to the Gemini API to analyze underlying waste and generate deep contextual text logs.

ID	Requirement Name	Description
FR-06	Personalized Budget Advice	AI-driven recommendation engines must evaluate current month trends against fixed goals and generate proactive advice prompts within the interface.
FR-07	Interactive Data Visualization	The dashboard must compile monthly figures and render dynamic analytical interfaces including pie charts for category distributions and bar charts for cash flows.
FR-08	Smart Notification Dispatcher	The application must check system variables and dispatch notifications for budget threshold limit breaches, upcoming structural bills, and automated tips.
FR-09	Predictive Budget Forecasting	The system must apply analytical algorithms using historical records to compute and display a baseline spending forecast model for the subsequent calendar month.
FR-10	Profile Modification Control	Users must be able to securely update registration profiles, modify password keys, reset default operational currencies, and adjust monthly savings benchmarks.

3. Non-Functional Requirements

Non-functional requirements characterize operational standards, quality boundaries, security frameworks, and technical system constraints.

ID	Requirement Quality	Target Threshold Specification
NFR-01	Performance & Latency	Standard app screens and charts must load internal database metrics in under 3.0 seconds. AI-based processing calls to external APIs must yield structural advice packages in under 7.0 seconds.
NFR-02	Interface Responsiveness	The frontend built using Flutter must render adaptable layouts that fluidly resize across varied mobile device screen formats, aspect ratios, and orientations
NFR-03	Data Encryption Standard	All user-specific passwords stored inside MongoDB must be cryptographically hashed using salting techniques via algorithms like bcrypt.
NFR-04	Session Communications Security	All transit-level communications across the mobile client and Node.js backend endpoints must operate over encrypted HTTPS connections using TLS protocols.
NFR-05	Scalability Capacity	The backend server runtime layer must support up to 1000 concurrent transactional logging requests without service crash or performance degradation.
NFR-06	Reliability and Availability	The storage engines and cloud platform hosting must maintain an operational service runtime availability up-time metric exceeding 99.5% over any calendar month

4. User Roles

The system interactions are divided into separate standard user personas, mapped completely to distinct operational privileges:

4.1 End-User / Client

Responsibilities: Manages personal profile configuration, inputs structural income/expense logs, creates localized budget restrictions, triggers automated AI analytics reports, views historical data visualizations, and modifies preference flags.

5. Use Cases

This section details structural transactional behaviors mapping out precise workflows across actors and system responses.

Use Case 01: User Authentication & Token Generation

Actor: End-User

Preconditions: The user possesses an active network connection and a previously registered profile within the system data store.

Action:

1. The End-User launches the application and types their registered email and secure password into the login interface.
2. The user clicks the "Login" button submission interface.
3. The system formats the data block and passes it to the secure Node.js server.

System Response: The backend verifies the credential hashes against MongoDB records, generates a signed JSON Web Token (JWT), transmits the key token back to the Flutter frontend client, and opens up the user's customized interactive dashboard view.

Use Case 02: Transaction Logging & Category Tagging

Actor: End-User

Preconditions: User must be authenticated and viewing the operational ledger interface.

Action:

1. The user clicks the "Add Transaction" float icon.
2. The user keys in the numerical amount, sets the transactional date calendar object, and types a short text description (e.g., "Dinner at restaurant").
3. The user hits the "Save Transaction" confirmation action.

System Response: The application auto-detects contextual categories, stores the transaction entry directly to MongoDB via the backend, updates the UI chart visuals immediately, and displays a success alert toast banner.

Use Case 03: Request AI-Driven Financial Insight Reports

Actor: End-User

Preconditions: User must have logged transaction items spanning at least one calendar week.

Action:

1. The user navigates to the "AI Analytics Tab" on the mobile dashboard.
2. The user initiates the transaction audit request by selecting the "Generate AI Report" control element.

System Response: The server aggregates the raw historical rows, wraps them securely inside an API request payload, queries the external Gemini API models, extracts the textual contextual output regarding overspending behaviors, caches it locally, and projects the formatted actionable insight text card block within the Flutter UI layer.